

Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 8a - Model-free RL (Part 3)

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Gradient Descent

Gradient Descent

Given a function $f(\theta_1, \dots, \theta_n)$ of n real values $\theta = (\theta_1, \dots, \theta_n)$, we want to minimize f with respect to θ

Gradient Descent

- The **gradient of f at point θ** , denoted $\nabla f(\theta)$ is an n -dimensional vector that points in the direction where f increases most steeply at point θ .
- Calculus tells us that $\nabla f(\theta)$ is just a vector of partial derivatives

$$\nabla f(\theta) = \left[\frac{\partial f(\theta)}{\partial \theta_1}, \dots, \frac{\partial f(\theta)}{\partial \theta_n} \right]$$

$$\text{where } \frac{\partial f(\theta)}{\partial \theta_i} = \lim_{\epsilon \rightarrow 0} \frac{f(\theta_1, \dots, \theta_{i-1}, \theta_i + \epsilon, \theta_{i+1}, \dots, \theta_n) - f(\theta)}{\epsilon}$$

We can decrease f by moving in negative gradient direction

Gradient Descent

Policy-based RL

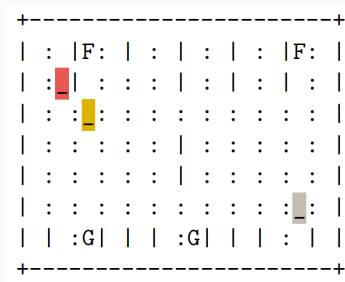
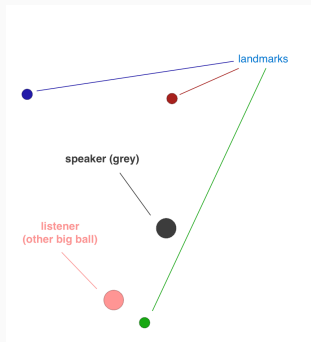
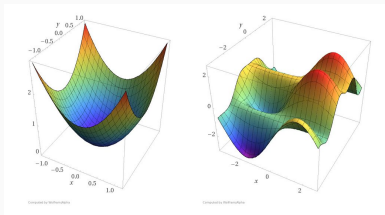
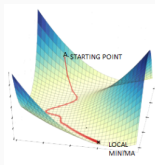
Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Gradient Descent



Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Visual Example: Gradient Descent

Suppose we want to minimize

$$f(\theta) = (\theta - 3)^2.$$

The gradient is

$$\frac{df}{d\theta} = 2(\theta - 3).$$

Start with

$$\theta = 0.$$

Then

$$\nabla f(0) = -6.$$

Gradient descent performs

$$\theta \leftarrow 0 - \alpha(-6) = 0 + 6\alpha.$$

The parameter moves toward the



Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Gradient Descent for Squared Error

- Suppose that we have a sequence of states and target values for each state $\langle s_1, V(s_1) \rangle, \langle s_2, V(s_2) \rangle, \dots$
 - for instance, produced by TD-based RL loop
- Our goal is to ?

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Gradient Descent for Squared Error

- Suppose that we have a sequence of states and target values for each state $\langle s_1, V(s_1) \rangle, \langle s_2, V(s_2) \rangle, \dots$
 - for instance, produced by TD-based RL loop
- Our goal is to $\rightarrow$ minimize the sum of squared errors between our estimated function and each target value:

$$\mathbb{E}_j = \frac{1}{2}(V_\theta(s_j) - v(s_j))^2$$

where

- $\mathbb{E}_j$ is the squared error of example j
- $V_\theta(s_j)$ is our estimated value for s_j
- $v(s_j)$ is the target of s_j
- After seeing s_j ?

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Gradient Descent for Squared Error

- Suppose that we have a sequence of states and target values for each state $\langle s_1, V(s_1) \rangle, \langle s_2, V(s_2) \rangle, \dots$
 - for instance, produced by TD-based RL loop
- Our goal is to $\rightarrow$ minimize the sum of squared errors between our estimated function and each target value:

$$\mathbb{E}_j = \frac{1}{2}(V_\theta(s_j) - v(s_j))^2$$

where

- $\mathbb{E}_j$ is the squared error of example j
- $V_\theta(s_j)$ is our estimated value for s_j
- $v(s_j)$ is the target of s_j
- After seeing $s_j \rightarrow$ the **gradient descent rule** tells us that we can decrease error by updating parameters by:

$$\theta_i := \theta_i - \alpha \frac{\partial \mathbb{E}_j}{\partial \theta_i}$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Aside: continued ...

$$\begin{aligned}\theta_i &\leftarrow \theta_i - \alpha \frac{\partial \mathbb{E}_j}{\partial \theta_i} \\&= \theta_i - \alpha \frac{\partial \mathbb{E}_j}{\partial V_\theta(s_j)} \frac{\partial V_\theta(s_j)}{\partial \theta_i} \\&= \theta_i - \alpha (V_\theta(s_j) - v(s_j)) \frac{\partial V_\theta(s_j)}{\partial \theta_i} \\&\stackrel{linear}{=} \theta_i - \alpha (V_\theta(s_j) - v(s_j)) f_i(s_j)\end{aligned}$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Aside: continued ...

$$\begin{aligned}\theta_i &\leftarrow \theta_i - \alpha \frac{\partial \mathbb{E}_j}{\partial \theta_i} \\&= \theta_i - \alpha \frac{\partial \mathbb{E}_j}{\partial V_\theta(s_j)} \frac{\partial V_\theta(s_j)}{\partial \theta_i} \\&= \theta_i - \alpha (V_\theta(s_j) - v(s_j)) \frac{\partial V_\theta(s_j)}{\partial \theta_i} \\&\stackrel{\text{linear}}{=} \theta_i - \alpha (V_\theta(s_j) - v(s_j)) f_i(s_j)\end{aligned}$$

- Thus the update becomes: $\theta_i := \theta_i + \alpha (v(s_j) - V_\theta(s_j)) f_i(s_j)$
- (For linear functions) this update is guaranteed to converge to best approximation for suitable learning rate schedule

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Policy-based RL

From Value-Based to Policy-Based RL

In value-based methods, we learn a value function:

$$V^{\pi}(s) \quad \text{or} \quad Q^{\pi}(s, a)$$

and then derive a policy, for example:

$$\pi(s) = \arg \max_a Q(s, a)$$

In policy-based methods, we learn the policy directly:

$$\pi_{\theta}(a \mid s)$$

Instead of learning how good actions are, learn how to act.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Reminder: Policy

- A **deterministic** policy is a mapping:
 - $\pi : \mathcal{S} \rightarrow \mathcal{A}$ from states to actions
 - $\pi : \mathcal{B} \rightarrow \mathcal{A}$ from beliefs to action
 - $\pi(s) / \pi(\beta)$ is the (single) action the agent will perform at state s or belief β .
- A **non-deterministic** policy is a mapping:
 - $\pi : \mathcal{S} \rightarrow \mathcal{A}^n$ from states to actions,
 - $\pi : \mathcal{B} \rightarrow \mathcal{A}^n$ from beliefs to actions
 - $\pi(s) / \pi(\beta)$ is the set of actions, one of which the agent will perform at state s or belief β .
- A **stochastic** policy is a mapping:
 - $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ from states to actions
 - $\pi : \mathcal{B} \times \mathcal{A} \rightarrow [0, 1]$ from beliefs to action
 - $\pi(s, a) / \pi(\beta, a)$ is the probability the agent will perform action a at state s / belief β .

Why do we need a **probabilistic** policy?

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Deterministic vs. Stochastic Policies

A deterministic policy maps states directly to actions:

$$a = \pi(s).$$

A stochastic policy defines a distribution over actions:

$$\pi(a \mid s).$$

Example:

$$\pi(\text{left} \mid s) = 0.4, \quad \pi(\text{right} \mid s) = 0.6.$$

Policy-gradient methods usually optimize stochastic policies.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Why Learn the Policy Directly?

Policy-based methods are especially useful when:

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Why Learn the Policy Directly?

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Policy-based methods are especially useful when:

- The action space is continuous.
- The optimal behavior should be stochastic.
- Computing $\arg \max_a Q(s, a)$ is difficult.
- We want smooth changes in behavior.
- We use neural networks to output action distributions.

Parameterized Policies

We represent the policy using parameters θ :

$$\pi_{\theta}(a \mid s)$$

The parameters may be:

- entries in a table,
- weights of a linear model,
- weights of a neural network.

The learning problem becomes:

Find parameters θ that produce high-return behavior.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Example: Softmax Policy

Assume one state and two actions:

$$a \in \{L, R\}.$$

Let the policy use action preferences:

$$\theta = \begin{bmatrix} \theta_L \\ \theta_R \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

The softmax policy is:

$$\pi_{\theta}(a) = \frac{e^{\theta_a}}{e^{\theta_L} + e^{\theta_R}}.$$

Therefore:

$$\pi_{\theta}(L) = \frac{e^2}{e^2 + e^1} \approx 0.73, \quad \pi_{\theta}(R) = \frac{e^1}{e^2 + e^1} \approx 0.27.$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Policy Gradient

The Policy Objective

We want to maximize expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau)].$$

A trajectory is:

$$\tau = s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T.$$

The discounted return from time t is:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}.$$

Learning means changing θ to increase $J(\theta)$.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Because we want to maximize $J(\theta)$, we use gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta).$$

Alternatively, we might aim at minimizing a loss measure:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta).$$

Move parameters in the direction that increases expected return.

The Key Identity

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

The policy-gradient trick uses:

$$\nabla_{\theta} \pi_{\theta}(a \mid s) = \pi_{\theta}(a \mid s) \nabla_{\theta} \log \pi_{\theta}(a \mid s).$$

This lets us write the gradient of expected return as:

$$\nabla_{\theta} J(\theta) = \mathbb{E} [G_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t)].$$

Increase the probability of actions that produced high return.

Policy-Gradient Update Rule

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

For one sampled transition in an episode:

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

Interpretation:

- If G_t is large and positive, make a_t more likely in s_t .
- If G_t is negative, make a_t less likely in s_t .
- The term $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ tells us how to change parameters to affect the selected action.

Example: Setup

We use the same softmax policy:

$$\theta = \begin{bmatrix} \theta_L \\ \theta_R \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

So:

$$\pi(L) = 0.73, \quad \pi(R) = 0.27.$$

Assume the agent samples action $a = R$ and later receives return:

$$G = 8.$$

How should the policy change?

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Example: Gradient of a Softmax Log-Policy

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

For a softmax policy, if action a was selected:

$$\frac{\partial \log \pi(a)}{\partial \theta_i} = \begin{cases} 1 - \pi(i), & i = a, \\ -\pi(i), & i \neq a. \end{cases}$$

Here, the selected action is R .

Therefore:

$$\nabla_{\theta} \log \pi(R) = \begin{bmatrix} -\pi(L) \\ 1 - \pi(R) \end{bmatrix} = \begin{bmatrix} -0.73 \\ 0.73 \end{bmatrix}.$$

Example: Compute the Gradient Update

The policy-gradient update is:

$$\theta \leftarrow \theta + \alpha G \nabla_{\theta} \log \pi(R).$$

Use:

$$\alpha = 0.1, \quad G = 8.$$

Then:

$$G \nabla_{\theta} \log \pi(R) = 8 \begin{bmatrix} -0.73 \\ 0.73 \end{bmatrix} = \begin{bmatrix} -5.84 \\ 5.84 \end{bmatrix}.$$

So:

$$\theta_{new} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 0.1 \begin{bmatrix} -5.84 \\ 5.84 \end{bmatrix} = \begin{bmatrix} 1.416 \\ 1.584 \end{bmatrix}.$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Example: Effect on the Policy

Before the update:

$$\theta = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \Rightarrow \pi(L) = 0.73, \quad \pi(R) = 0.27.$$

After the update:

$$\theta = \begin{bmatrix} 1.416 \\ 1.584 \end{bmatrix}.$$

New softmax probabilities:

$$\pi(L) \approx 0.46, \quad \pi(R) \approx 0.54.$$

The successful action R became more likely.

Gradient Descent

Policy-based RL

Policy Gradient

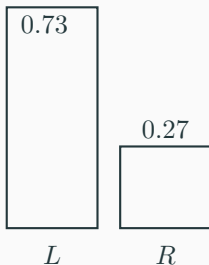
REINFORCE

Actor-Critic

Conclusion

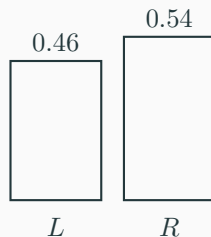
Example: Visualization of the Policy Change

Before update



update
→

After update



The gradient does not directly edit probabilities. It changes parameters, and the new parameters induce new probabilities.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

What if the Return Is Negative?

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Suppose the same action R was selected, but now:

$$G = -6.$$

The update becomes:

$$\theta \leftarrow \theta + \alpha(-6)\nabla_{\theta} \log \pi(R).$$

This moves parameters in the opposite direction.

Positive return reinforces the action.
Negative return discourages the action.

REINFORCE

Policy Gradient: A General Framework

How can we use policy gradients to learn a policy?

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Policy Gradient: A General Framework

How can we use policy gradients to learn a policy?

All policy-gradient methods follow the same basic idea:

At every iteration:

- 1 Execute the current policy

$$\pi_{\theta}(a|s)$$

- 2 Collect trajectories

$$\tau = s_0, a_0, r_1, \dots$$

- 3 Estimate how good each selected action was

$$G_t, \quad Q(s, a), \quad A(s, a)$$

- 4 Update the policy

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Different policy-gradient algorithms differ only in Step 3.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Policy Gradient: A General Framework

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

There is a variety of approaches. The main difference between algorithms is in **how they estimate the quality of each action.**

Algorithm	Uses
REINFORCE	Monte-Carlo return G_t
Actor-Critic	TD error / Advantage
PPO	Clipped advantage estimate
SAC	Soft Q-value + entropy

First Policy-Gradient Algorithm: REINFORCE

REINFORCE is the simplest policy-gradient algorithm.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

First Policy-Gradient Algorithm: REINFORCE

REINFORCE is the simplest policy-gradient algorithm.

It answers Step 3 by using the Monte-Carlo return:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

First Policy-Gradient Algorithm: REINFORCE

REINFORCE is the simplest policy-gradient algorithm.

It answers Step 3 by using the Monte-Carlo return:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$$

The update becomes

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

First Policy-Gradient Algorithm: REINFORCE

REINFORCE is the simplest policy-gradient algorithm.

It answers Step 3 by using the Monte-Carlo return:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$$

The update becomes

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

REINFORCE = policy gradient + Monte-Carlo estimate of action quality.

It is called a Monte Carlo method because G_t is computed from sampled returns.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

REINFORCE

- 1: Initialize policy parameters θ
 - 2: **while** not converged **do**
 - 3: Generate an episode using π_θ
 - 4: **for** each time step t in the episode **do**
 - 5: Compute return $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$
 - 6: $\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(a_t | s_t)$
 - 7: **end for**
 - 8: **end while**
-

The Main Problem: High Variance

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Policy-gradient estimates can be very noisy.

Two trajectories may start similarly but receive very different returns:

$$G = 10, \quad 8, \quad -4, \quad 12, \quad -7, \quad 9.$$

This causes unstable gradient estimates.

REINFORCE is simple and unbiased, but often high variance.

Variance Reduction with a Baseline

We can subtract a baseline $b(s_t)$:

$$\nabla_{\theta} J(\theta) = \mathbb{E} [(G_t - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] .$$

A common choice is:

$$b(s_t) = V(s_t).$$

Then:

$$G_t - V(s_t)$$

is an estimate of the advantage.

The baseline reduces variance without changing the expected gradient.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Actor-Critic

From REINFORCE to Actor-Critic

REINFORCE estimates the return using complete episodes.

Limitations:

- Must wait until the episode terminates.
- High-variance Monte Carlo estimates.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

From REINFORCE to Actor-Critic

REINFORCE estimates the return using complete episodes.

Limitations:

- Must wait until the episode terminates.
- High-variance Monte Carlo estimates.

Instead, we can learn a function that estimates the return while interacting with the environment.

Actor-Critic methods learn two functions:

- **Actor:** policy $\pi_{\theta}(a|s)$
- **Critic:** value function $V_w(s)$ (or $Q_w(s, a)$)

The critic evaluates actions while the actor learns the policy.

Gradient Descent

Policy-based RL

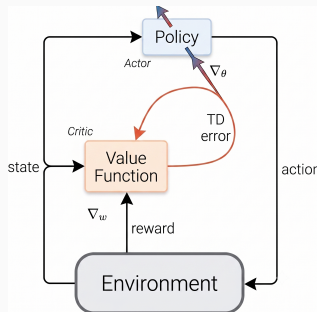
Policy Gradient

REINFORCE

Actor-Critic

Conclusion

How Actor-Critic Works



Gradient Descent

Policy-based RL

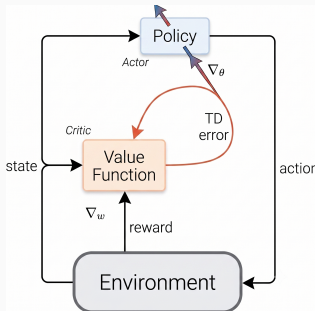
Policy Gradient

REINFORCE

Actor-Critic

Conclusion

How Actor-Critic Works



The interaction loop is:

- 1 Actor selects an action.
- 2 Environment returns reward and next state.
- 3 Critic evaluates the outcome.
- 4 Actor updates the policy using the critic's feedback.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

How Actor-Critic Works

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Adding a value-function baseline to REINFORCE already gives an Actor-Critic architecture.

However, if the critic is still trained from complete Monte Carlo returns, we still suffer from delayed updates.

A common approach is to train the critic with TD learning, allowing updates after every interaction.

Actor-Critic Algorithm

One-Step TD Actor-Critic

- 1: Initialize actor parameters θ
 - 2: Initialize critic parameters w
 - 3: **while** not converged **do**
 - 4: Observe current state s
 - 5: Sample action $a \sim \pi_{\theta}(\cdot|s)$
 - 6: Execute action
 - 7: Observe reward r and next state s'
 - 8: Compute TD error
$$\delta = r + \gamma V_w(s') - V_w(s)$$
 - 9: Update critic
$$w \leftarrow w + \beta \delta \nabla_w V_w(s)$$
 - 10: Update actor
$$\theta \leftarrow \theta + \alpha \delta \nabla_{\theta} \log \pi_{\theta}(a|s)$$
 - 11: **end while**
-

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Why Use the Advantage?

Suppose the critic predicts $V(s) = 8$.

This means that, on average, we expect a return of 8.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Why Use the Advantage?

Suppose the critic predicts $V(s) = 8$.

This means that, on average, we expect a return of 8.

Now suppose taking action a actually gives $Q(s, a) = 11$.

Then $A(s, a) = Q(s, a) - V(s) = 3$ (the action was better than expected).

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Why Use the Advantage?

Suppose the critic predicts $V(s) = 8$.

This means that, on average, we expect a return of 8.

Now suppose taking action a actually gives $Q(s, a) = 11$.

Then $A(s, a) = Q(s, a) - V(s) = 3$ (the action was better than expected).

Alternatively, $Q(s, a) = 6$ gives $A(s, a) = -2$, i.e., the action was worse than expected.

The advantage measures whether an action performed better or worse than expected.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Where Does the Advantage Estimate Come From?

The actor should update according to the advantage

$$A(s, a) = Q(s, a) - V(s).$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Where Does the Advantage Estimate Come From?

The actor should update according to the advantage

$$A(s, a) = Q(s, a) - V(s).$$

The critic does not know the true action value $Q(s, a)$.

Instead, it approximates it using one-step TD:

$$Q(s, a) \approx r + \gamma V(s').$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Where Does the Advantage Estimate Come From?

The actor should update according to the advantage

$$A(s, a) = Q(s, a) - V(s).$$

The critic does not know the true action value $Q(s, a)$.

Instead, it approximates it using one-step TD:

$$Q(s, a) \approx r + \gamma V(s').$$

Therefore,

$$A(s, a) \approx r + \gamma V(s') - V(s).$$

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Where Does the Advantage Estimate Come From?

The actor should update according to the advantage

$$A(s, a) = Q(s, a) - V(s).$$

The critic does not know the true action value $Q(s, a)$.

Instead, it approximates it using one-step TD:

$$Q(s, a) \approx r + \gamma V(s').$$

Therefore,

$$A(s, a) \approx r + \gamma V(s') - V(s).$$

This TD estimate replaces the Monte Carlo return in REINFORCE, enabling updates after every interaction.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Advantage Actor-Critic (A2C)

A2C combines:

- an **actor** that learns the policy,
- a **critic** that estimates the value function.

Instead of waiting until the end of an episode, the actor updates after every interaction using the critic's estimate of the advantage:

$$\hat{A}_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

The actor update is

$$\theta \leftarrow \theta + \alpha \hat{A}_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

The critic provides immediate feedback, greatly reducing variance compared to REINFORCE.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Advantage Actor-Critic (A2C)

One-Step Advantage Actor-Critic

- 1: Initialize actor parameters θ
 - 2: Initialize critic parameters w
 - 3: **while** training **do**
 - 4: Observe state s
 - 5: Sample action $a \sim \pi_{\theta}(\cdot|s)$
 - 6: Execute a
 - 7: Observe reward r and next state s'
 - 8: Compute
$$\delta = r + \gamma V_w(s') - V_w(s)$$
 - 9: Critic update
$$w \leftarrow w + \beta \delta \nabla_w V_w(s)$$
 - 10: Actor update
$$\theta \leftarrow \theta + \alpha \delta \nabla_{\theta} \log \pi_{\theta}(a|s)$$
 - 11: **end while**
-

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Why Isn't A2C Enough?

Although A2C is much more efficient than REINFORCE, it still has a problem.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Why Isn't A2C Enough?

Although A2C is much more efficient than REINFORCE, it still has a problem.

Large policy updates may change the policy too much.

This can cause

- unstable learning,
- catastrophic drops in performance,
- oscillation between good and bad policies.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Why Isn't A2C Enough?

Although A2C is much more efficient than REINFORCE, it still has a problem.

Large policy updates may change the policy too much.

This can cause

- unstable learning,
- catastrophic drops in performance,
- oscillation between good and bad policies.

Can we improve the policy while preventing excessively large updates?

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Proximal Policy Optimization (PPO)

PPO is an Actor-Critic algorithm with one additional idea:

Don't let the policy change too much in a single update.

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Proximal Policy Optimization (PPO)

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

PPO is an Actor-Critic algorithm with one additional idea:

Don't let the policy change too much in a single update.

Instead of taking arbitrarily large gradient steps,

PPO clips the policy update so that

$$\pi_{\text{new}} \approx \pi_{\text{old}}.$$

Many small, safe improvements are better than a few large, unstable ones.

Repeat:

- 1 Execute the current policy.
- 2 Estimate the advantage.
- 3 Compute the policy gradient.
- 4 Clip excessively large policy updates.
- 5 Update the actor and critic.

Typical objective:

$$L^{PPO} = \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right),$$

where

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}.$$

Clipping keeps policy updates stable while still improving performance.

Evolution of Policy-Gradient Methods

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Method	Uses	Update	Variance
REINFORCE	Monte Carlo Return	Episode End	High
Actor-Critic	TD Error	Every Step	Lower
A2C	Advantage Estimate	Every Step	Lower
PPO	Clipped Advantage	Every Step	Lower + Stable

Each algorithm improves the estimate of the policy gradient and makes learning more stable.

Conclusion

Value-Based vs. Policy-Based Methods

Value-Based	Policy-Based
Learn V or Q	Learn π_θ directly
Policy derived by maximization	Policy is the learned object
Natural for discrete actions	Natural for continuous actions
Often lower variance	Often higher variance
Can be off-policy	Often on-policy

Gradient Descent

Policy-based RL

Policy Gradient

REINFORCE

Actor-Critic

Conclusion

Many modern deep RL algorithms build on this idea:

- REINFORCE
- Actor-Critic
- A2C / A3C
- TRPO
- PPO
- SAC

Policy gradients are central to modern continuous-control RL.